

Maternas — Informe Técnico de Desarrollo



**Institución
Universitaria
de Envigado**

VIGILADA MINEDUCACIÓN

Chatbot RAG para Información y Orientación en Salud Materna

Informe técnico del sistema Convocatoria **890 de Minciencias** · Institución Universitaria de Envigado

Equipo de desarrollo

- **Juan Pablo Ríos Ortiz** — `jpriosos@correo.iue.edu.co`
- **Daniel Restrepo Villa** — `drestrepov@correo.iue.edu.co`
- **Cristian Troncoso Guerra** — `ctroncosog@correo.iue.edu.co`

✚ Información del proyecto

Campo	Detalle
Proyecto	Agente inteligente basado en procesamiento de lenguaje natural para seguimiento materno en época pospandémica para un entorno de Telemedicina
Código del proyecto	COD_00-215
Institución	Institución Universitaria de Envigado
Versión evaluada	<code>754ef4a</code> · rama <code>master</code>
LLM Generador	<code>openai/gpt-oss-120b</code> (Groq) — migrado desde <code>llama-3.3-70b-versatile</code> , dado de baja por Groq el 16/08/2026
Índice en producción	Configuración D
Vectores indexados	253.455 (al momento de la evaluación en vivo, sección 6; 253.469 al momento de esta revisión, por documentos subidos durante pruebas posteriores)
Fecha del informe	25 de agosto de 2026 (revisión de contenido — la evaluación en vivo de la sección 6.3–6.9 sigue siendo la del 19 de agosto; ver nota en la sección 6)

1. Resumen ejecutivo

Maternas es un chatbot conversacional basado en Recuperación Aumentada por Generación (RAG) orientado a madres gestantes y en período postparto, que responde consultas de salud materna en español —control prenatal, signos de alarma, medicamentos, nutrición, lactancia y salud mental perinatal— clasificando primero la intención y el riesgo clínico de cada mensaje antes de generar una respuesta fundamentada en literatura médica indexada. El sistema opera con costo marginal cercano a cero (APIs gratuitas de Groq/Cerebras, embedding local) sobre hardware de gama media, sin fine-tuning. A la fecha de este informe el desarrollo está en estado **funcional y probado en vivo**: expone tres interfaces (API FastAPI, UI Streamlit con panel de administración, bot de Telegram), un índice FAISS de 253.455 vectores en producción (Config D), una suite de 5 configuraciones de retrieval evaluadas experimentalmente con Ragas, y una batería de pruebas automatizadas (`pytest`). El LLM generador se migró de `llama-3.3-70b-versatile` a `openai/gpt-oss-120b` tras la baja del primero por parte de Groq (16/08/2026); esta versión del informe reporta el sistema ya migrado y revalidado, tanto en vivo (sección 6) como con una nueva corrida de evaluación (sección 8, mismo protocolo y semilla que la corrida anterior). Las métricas de evaluación actuales (`faithfulness` 0.392, `answer_correctness` 0.636, `answer_relevancy` 0.810 sobre baseline 0.558) muestran una recuperación de contexto todavía por debajo del sistema de referencia publicado, atribuible principalmente a la ausencia de los documentos fuente exactos del benchmark de evaluación en el índice de producción — una limitación conocida y documentada, no un hallazgo nuevo de este informe. `answer_relevancy` y `answer_correctness` mejoraron de forma notable respecto a la corrida anterior; `faithfulness` bajó y la latencia subió — ambos atribuibles al overhead de razonamiento del nuevo modelo (detalle en sección 8). Tras esa evaluación en vivo (19/08/2026) el desarrollo continuó con tres entregas adicionales, documentadas en las nuevas secciones 6.10–6.12 de esta revisión: una subsección de **uso en tiempo real** anonimizado en el panel de Métricas (sesiones activas de Streamlit y Telegram, sin ningún identificador que permita distinguir una de otra); una corrección al **notificador de riesgo** para que el correo de alerta incluya la secuencia completa de mensajes que llevaron a esa alerta y para dejar de reenviar el mismo aviso en turnos posteriores ya no relacionados; y una **segunda capa de aceptación** (Términos y Condiciones, borrador pendiente de revisión legal) que se muestra tras el aviso de tratamiento de datos, en Streamlit y en Telegram.

2. Contexto y objetivos

Problema que resuelve

En contextos de bajos recursos, las gestantes frecuentemente no tienen acceso rápido a orientación médica ante dudas cotidianas o síntomas de alarma. Maternas actúa como primer filtro informativo: clasifica la urgencia clínica de cada consulta y, cuando detecta riesgo medio o alto, escala mediante una notificación automática por correo electrónico, además de indicar siempre al usuario que busque atención profesional.

Alcance funcional

- Conversación en lenguaje natural sobre síntomas del embarazo, control prenatal, medicamentos, nutrición, lactancia, postparto y salud mental perinatal.
- Clasificación automática de intención (12 categorías) y de riesgo clínico (bajo/medio/alto, en cascada heurística → LLM).
- Recuperación de fragmentos de literatura médica (FAISS) y generación de respuestas con citas a la fuente.
- Escalamiento por correo electrónico ante riesgo medio/alto.
- Panel de administración para gestión del índice, configuración en caliente y monitoreo.

Usuarios objetivo

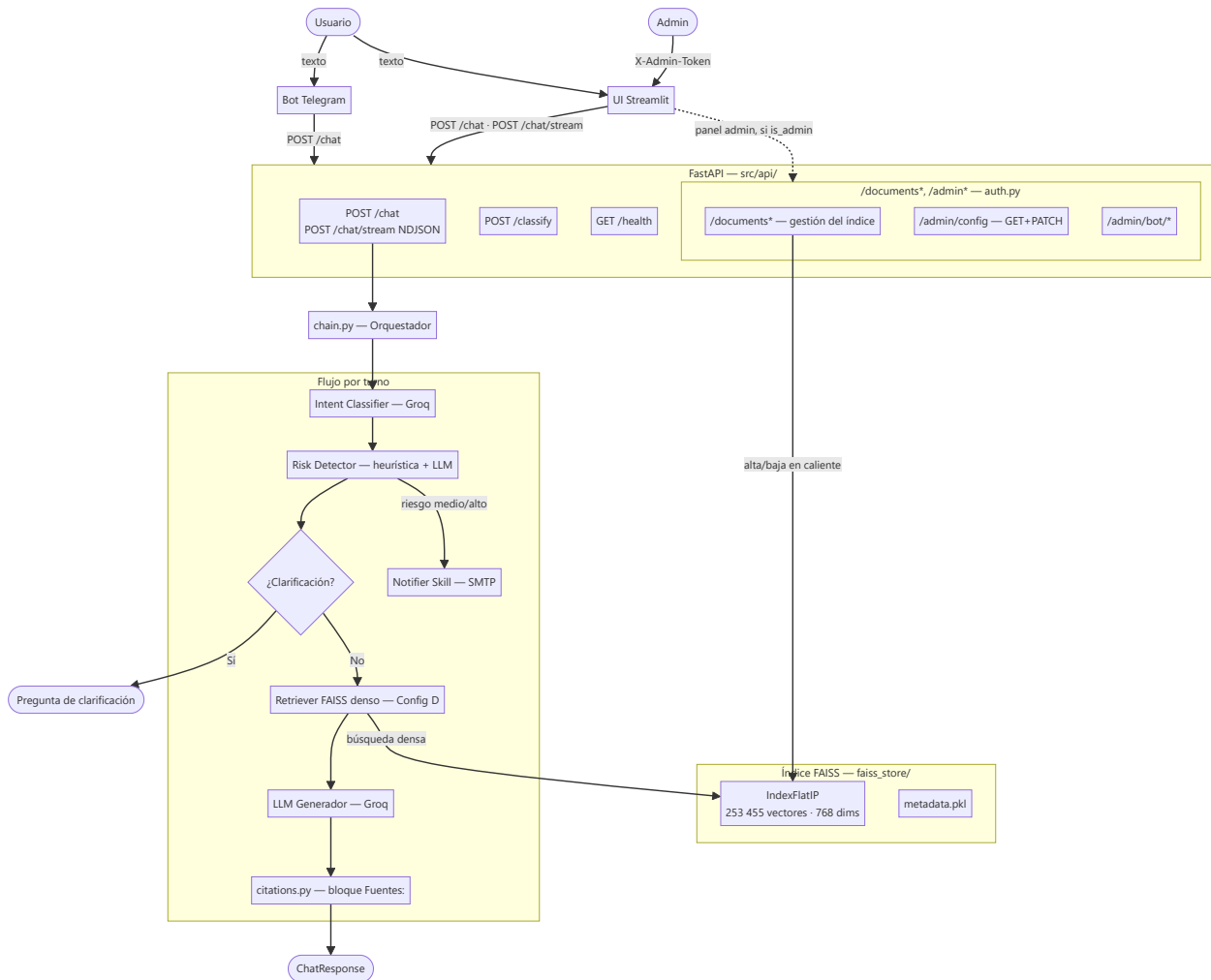
Gestantes y su red de apoyo (pareja, familiares) — el sistema está diseñado explícitamente para que terceros puedan consultar en representación o junto a la gestante (sin restricción de autoría en el prompt ni en la UI; no se incluye un caso de prueba específico de este escenario en la sección 6 de esta versión del informe).

Restricciones del proyecto

Según `foragents/project_constraints.md`: sin QLoRA ni fine-tuning en esta fase, priorizando simplicidad, rapidez de implementación y costo cercano a cero, sobre hardware disponible (AMD Ryzen 5, RTX 2050, 16 GB RAM).

3. Arquitectura de la solución

Diagrama de alto nivel



Componentes principales

Componente	Responsabilidad
Bot Telegram	Cliente ligero: reenvía a <code>POST /chat</code> , historial en RAM, scheduler de check-ins
UI Streamlit	Chat público + panel admin (Dashboard/Documentos/Métricas/Configuración/Consola)
FastAPI	Valida requests, carga FAISS en <code>lifespan</code> , expone endpoints públicos y administrativos
chain.py	Orquestador del turno: intent → risk → clarificación → notificación → retrieval → generación → citas
Intent Classifier	12 categorías, zero-shot vía Groq, con fallback heurístico

Componente	Responsabilidad
Risk Detector	3 niveles, heurística instantánea + LLM de confirmación, consciente del historial de la conversación
Retriever (Config D)	Búsqueda densa FAISS pura sobre <code>medmcqa</code> + <code>medqa_*</code> + <code>maternaqaes_lm</code>
Notifier Skill	Alerta SMTP ante riesgo medio/alto

Arquitecturas evaluadas

El retriever pasó por cinco configuraciones sucesivas, cada una medida con el mismo protocolo de evaluación Ragas antes de decidir el cambio (detalle completo en `foragents/retrieval_arquitecturas_configs.md` y `foragents/qa_technical.md`):

Config	Descripción	Resultado de la decisión
A — FAISS puro	Búsqueda densa sobre todo el índice sin distinción de fuente	Descartada: casos clínicos de Multiclinsum contaminaban el contexto
B — FAISS + BM25	Capa densa filtrada por fuente + BM25 léxico solo para Multiclinsum	Mejoró <code>faithfulness</code> (+41%) y latencia (−9%) vs A
C — + corpus obstétrico ES	Se añadió <code>maternaqaes_lm</code> (corpus colombiano)	Salto grande en <code>context_recall</code> (0.067→0.452) y <code>faithfulness</code> (+27%)
D — sin <code>textbook</code> / <code>multiclinsum</code> (<i>producción actual</i>)	Se removieron 127.290 vectores (33,4% del índice) por riesgo de licencia no resuelto (18 textbooks EN sin licencia de reuso identificable, MultiClinSum sin garantía de desidentificación documentada)	Medido primero contra C antes de ejecutar: deltas dentro del ruido estadístico (± 0.25 std, N=14) — sin pérdida medible de calidad. <code>bm25_index.py</code> se eliminó por quedar sin corpus que indexar
E — Config D + HyDE	Genera un párrafo hipotético vía LLM antes de embedder la query	Descartada. Ninguna métrica mejoró de forma concluyente (deltas dentro del ruido); costo real: +1,27 s de latencia por turno y agotamiento de cuota diaria de Groq a mitad de la corrida de evaluación

La decisión de remover `textbook` / `multiclinsum` (Config D) y de no adoptar HyDE (Config E) ilustra el patrón de trabajo del equipo: cada cambio de arquitectura se midió con Ragas sobre el mismo conjunto de pares *antes* de aplicarse en producción, en vez de decidirse a priori.

4. Stack tecnológico

Capa	Tecnología	Versión
Lenguaje	Python	3.12.7
API backend	FastAPI + uvicorn	0.115.6 / 0.32.1
Interfaz web	Streamlit	1.41.1
Bot de mensajería	python-telegram-bot	21.10
LLM generador	<code>openai/gpt-oss-120b</code> (Groq API)	—
LLM evaluador (judge)	<code>gemma-4-31b</code> (Cerebras API)	—
Embedding	<code>intfloat/multilingual-e5-base</code> (768 dims, ES/EN/ZH)	—
Vector store	FAISS <code>IndexFlatIP</code>	1.9.0
Orquestación LLM	LangChain (text splitters)	0.3.13
Evaluación	Ragas	0.2.12
Validación de configuración	Pydantic Settings	2.14.1
Cifrado de datos en reposo	<code>cryptography</code> (Fernet)	—

Datasets indexados: **MedMCQA** (187.005 preguntas médicas EN, Apache 2.0), **MedQA** (USMLE/Taiwan/Mainland, MIT) y **MaternaQA-es LM** (5.353 sub-chunks, corpus obstétrico colombiano, único dataset en español específico del dominio).

5. Setup

Pasos verificados para levantar el sistema en local (`README.md` y `docs/DOCUMENTACION.md` §14).

Instalación

```
# 1. Clonar el repositorio
git clone https://github.com/elrios893/maternas-rag.git
cd maternas-rag

# 2. Crear entorno virtual
python -m venv venv
.\venv\Scripts\activate      # Windows
# source venv/bin/activate   # Linux/macOS

# 3. Instalar PyTorch con soporte CUDA (si hay GPU NVIDIA – recomendado para ingesta)
pip install torch==2.5.1+cu121 --index-url https://download.pytorch.org/whl/cu121

# 4. Instalar el resto de dependencias
# (requirements.txt pinea sentence-transformers==2.7.0 – la versión 3.3.1
# producía un cuelgue silencioso al importar junto con torch)
pip install -r requirements.txt

# 5. Configurar variables de entorno
cp .env.example .env
# Editar .env con GROQ_API_KEY y demás valores requeridos
```

Arrancar el sistema

```
# Terminal 1 – API FastAPI
python -m uvicorn src.api.main:app --port 8080 --reload

# Terminal 2 – UI Streamlit
streamlit run src/ui/app.py

# Terminal 3 – Bot Telegram (opcional, requiere la API corriendo)
python src/bot/maternas_bot.py
```

- UI Streamlit: <http://localhost:8501>
- API docs (Swagger): <http://localhost:8080/docs>

Ingesta del índice FAISS (one-time, ~5h con GPU — no se ejecuta en este informe, ver sección 6.1)

```
python src/ingestion/run_ingestion.py
# o por dataset individual:
python -m src.ingestion.ingest_medmcqa
python -m src.ingestion.ingest_medqa
python -m src.ingestion.ingest_maternaqaes_lm
```

Tests

```
./venv/Scripts/python.exe -m pytest -q
```

Suite en `tests/` (clasificadores, gestión de documentos, config editable, supervisor del bot, citas, chat streaming, usuarios activos). Los fixtures de `tests/conftest.py` evitan cargar el índice FAISS real (~780 MB) y resetean los singletons de clientes Groq cacheados entre tests.

6. Pruebas y resultados

Cada subproceso sigue el formato *narrativa corta* → *evidencia visual* → *observación técnica*. Las secciones 6.1 y 6.2 (ingestión, indexación) se documentan con el código real, sin ejecución en vivo (no dependen del LLM generador y no cambiaron con la migración). Las secciones 6.3 en adelante corresponden a la evaluación en vivo del **19 de agosto de 2026**, ya con `openai/gpt-oss-120b` como LLM de producción, contra la API (`localhost:8080`), la UI Streamlit (`localhost:8501`) y el bot de Telegram real (`@MaternasAssistant_bot`), con el índice FAISS de producción (253.455 vectores) cargado. Para cada caso de prueba se abrió una sesión nueva (pestaña nueva de Streamlit, o `/reset` en Telegram), de modo que el aviso de tratamiento de datos y el estado de riesgo partieran limpios en cada uno. Las secciones 6.10–6.13 son nuevas en esta revisión (25/08/2026): documentan tres entregas posteriores a esa evaluación en vivo (uso en tiempo real, deduplicación del notificador de riesgo, segunda capa de Términos y Condiciones) con el mismo criterio que 6.1–6.2 — código real más la suite `pytest`, **sin una nueva ronda de capturas de pantalla** (no se re-ejecutó la evaluación en vivo del 19/08 para esta revisión de contenido).

6.1 Ingestión

La ingestión puebla el índice FAISS a partir de los datasets crudos y **no se ejecuta en vivo** para este informe: reconstruir o alterar el índice de producción toma horas (~5h con GPU) y puede dejarlo en un estado inconsistente. Se documenta con el código real y los registros de ejecuciones anteriores.

`src/ingestion/run_ingestion.py` orquesta tres scripts idempotentes (Multiclinsum, MedMCQA, MedQA) que pueden relanzarse desde el último checkpoint si el proceso se interrumpe. Cada dataset pasa por

`formatters.py` (7 formateadores, uno por fuente) y luego por `chunkers.py`, que decide la estrategia de chunking según `source_dataset`:

```

src/ingestion/run_ingestion.py

1 """run_ingestion.py - Orquestador de ingestión completa.
2
3 Ejecuta en orden:
4 1. Multiclinsum (summaries + fulltexts)
5 2. MedMCQA (train + dev)
6 3. MedQA (US + Taiwan + Mainland + textbooks EN)
7
8 Cada script es idempotente: si una fase ya fue completada la salta.
9 Si el proceso se interrumpe, al relanzar continúa desde el último checkpoint.
10 """
11
12 def main(only: str | None = None) -> None:
13     run_all = only is None
14     run_multiclin = run_all or only == "Multiclinsum"
15     run_medmcqa = run_all or only == "Medmcqa"
16     run_medqa = run_all or only == "Medqa"
17
18     if run_multiclin:
19         from src.ingestion.ingest_multiclinsum import main as run_multiclinsum
20         run_multiclinsum()
21
22     if run_medmcqa:
23         from src.ingestion.ingest_medmcqa import main as run_mmc
24         run_mmc()
25
26     if run_medqa:
27         from src.ingestion.ingest_medqa import main as run_mq
28         run_mq()
29
30     print("-" * 60)
31     print("INGESTIÓN COMPLETA")
32     print("-" * 60)

```

Captura del código real de `run_ingestion.py`: ejecuta los tres scripts de ingesta en orden y reporta el cierre del proceso.

```

src/ingestion/chunkers.py - dispatcher

1 CHUNK_SIZE_TOKENS = 400 # aprox. tokens por chunk (1 token = 4 chars)
2 CHUNK_OVERLAP_TOKENS = 80 # overlap entre chunks de textbooks
3
4 NO_CHUNK_SOURCES = {
5     "medmcqa", "medqa_us", "medqa_taiwan",
6     "medqa_mainland", "multiclinsum_summary",
7 }
8
9 def chunk_document(doc: Document) -> List[Document]:
10     """Decide qué estrategia aplicar según source_dataset."""
11     source = doc.metadata.get("source_dataset", "")
12
13     if source in NO_CHUNK_SOURCES:
14         return chunk_passthrough(doc)
15
16     if source == "multiclinsum_fulltext":
17         return chunk_paragraph_grouping(doc) # ~350 tok, overlap 1 párrafo
18
19     if source in ("textbook", "upload"):
20         return chunk_recursive_split(doc) # 400 tok / 80 overlap
21
22     return chunk_passthrough(doc) # fallback: source desconocido

```

`chunk_document()` decide entre passthrough (MedMCQA/MedQA, sin chunking), agrupación por párrafos (~350 tok, Multiclinsum fulltext) o `RecursiveCharacterTextSplitter` (400 tok / 80 overlap, textbooks/uploads), según `source_dataset`.

Observación técnica: el tamaño de chunk no es un parámetro arbitrario — la sección 8 de este informe muestra que re-chunkar `maternaqaes_1m` de ~879 a ~336 tokens promedio subió `faithfulness` un 170% (0,133→0,359), por lo que `CHUNK_SIZE_TOKENS=400` en `chunkers.py` refleja una decisión validada empíricamente, no un valor por defecto.

6.2 Indexación

Tampoco se ejecuta en vivo, por la misma razón que la ingestión. Se documenta con el código de `FAISSStore` y `embedder.py`.

```

src/ingestion/store.py - FAISSStore
1  """store.py - Constructor y lector del índice FAISS.
2
3  Archivos que gestiona en faiss_store/:
4  index.faiss      + vectores binarios
5  metadata.pkl     + dict { int_id + Document.metadata + text }
6  build_info.json  + auditoría: modelo, fecha, total de vectores
7  """
8
9  class FAISSStore:
10     """
11     Uso en ingestión:
12         store = FAISSStore.create_empty()
13         store.add_documents(chunks)
14         store.save()
15
16     Uso en retrieval:
17         store = FAISSStore.load()
18         results = store.search("¿qué es la preeclampsia?", k=5)
19     """
20
21     def is_active(meta: dict) -> bool:
22         """Un fragmento participa en el RAG salvo que esté
23         explícitamente desactivado.
24
25         El pipeline de ingestión NO escribe la clave 'active';
26         su ausencia significa ACTIVO. Nunca usar meta["active"]
27         sin default: descartaría el 100% del índice existente y
28         /chat seguiría respondiendo 200 con un fallback genérico,
29         sin ningún error visible.
30         """
31         return bool(meta.get("active", True))
32
33 src/ingestion/embedder.py - prefijos obligatorios (protocolo E5)
34
35 def embed_documents(texts: List[str], batch_size: int = 64, ...) -> np.ndarray:
36     """Aplica el prefijo "passage: " requerido por el modelo E5."""
37     model = _get_model()

```

`is_active()` — un fragmento participa en el RAG salvo que esté explícitamente desactivado; el docstring documenta por qué nunca se usa `meta["active"]` sin valor por defecto (descartaría el 100% del índice existente en silencio, ver también sección 9).

```

20
21 def is_active(meta: dict) -> bool:
22     """Un fragmento participa en el RAG salvo que esté
23     explícitamente desactivado.
24
25     El pipeline de ingestión NO escribe la clave 'active';
26     su ausencia significa ACTIVO. Nunca usar meta["active"]
27     sin default: descartaría el 100% del índice existente y
28     /chat seguiría respondiendo 200 con un fallback genérico,
29     sin ningún error visible.
30     """
31     return bool(meta.get("active", True))
32
33 src/ingestion/embedder.py - prefijos obligatorios (protocolo E5)
34
35 def embed_documents(texts: List[str], batch_size: int = 64, ...) -> np.ndarray:
36     """Aplica el prefijo "passage: " requerido por el modelo E5."""
37     model = _get_model()
38     prefixed = [f"passage: {t}" for t in texts]
39     ... # batches de 64, tqdm progress
40     matrix = np.vstack(all_vectors).astype(np.float32)
41     return _normalize(matrix) # l2 - coseno equivalente en IndexFlatIP
42
43 def embed_query(text: str) -> np.ndarray:
44     """Aplica el prefijo "query: " requerido por el modelo E5."""
45     model = _get_model()
46     prefixed = f"query: {text}"
47     vec = model.encode(prefixed, convert_to_numpy=True, normalize_embeddings=False)
48     return _normalize(vec.astype(np.float32).reshape(1, -1))[0]

```

`embed_documents()` antepone `"passage: "` y `embed_query()` antepone `"query: "` — requisito del modelo `multilingual-e5-base`; sin estos prefijos los scores de similitud bajan ~15% (`qa_technical.md` Q5).

Observación técnica: `FAISSStore` usa `IndexFlatIP` (búsqueda exacta, sin aproximación) en vez de `IndexIVFFlat`. Con ~253k–380k vectores la búsqueda exacta resuelve en 20–50ms; la aproximación de IVFFlat solo se justifica a partir de millones de vectores (ver decisión en sección 9).

6.3 Retrieval

Ejecutado en vivo contra el índice de producción a través de la UI Streamlit, ya con `openai/gpt-oss-120b` como LLM activo, en tres sesiones nuevas independientes: una gestante consultando directamente y continuando la conversación hacia un síntoma de riesgo medio, un flujo de clarificación completo, y un caso de riesgo alto directo.

Sesión nueva — aviso de tratamiento de datos:

The screenshot shows a dark-themed dialog box titled "Aviso sobre tratamiento de información". It contains the following text:

Debes aceptar el aviso de tratamiento de información para poder chatear.

AVISO SOBRE TRATAMIENTO DE INFORMACIÓN

Acepta el aviso de datos para poder escribirme

EM CHATGPT es un proyecto de INVESTIGACIÓN EN FASE EXPERIMENTAL. Una inteligencia artificial y NO SUSTITuye la orientación de un profesional competente.

Podemos conservar tu NOMBRE PREFERIDO O ALIAS y un CÓDIGO INTERNO de identificación. NO escribas tu nombre legal completo, número de documento, dirección, contraseñas ni datos financieros.

Tu conversación se procesa para mantener el CONTEXTO del diálogo y los flujos del proyecto de investigación. Si se usa en análisis o publicaciones, se aplican medidas de ANONIMIZACIÓN o SEUDONIMIZACIÓN.

Tu participación es VOLUNTARIA; puedes NO responder preguntas sensibles y puedes SOLICITAR EL RETIRO de tu información en cualquier momento.

Este es un PROTOTIPO DE INVESTIGACIÓN — aún no apto para uso clínico, comercial o productivo real.

¿Aceptas continuar bajo estas condiciones?

Buttons: ☐ Acepto ☒ No acepto

Cada sesión nueva de navegador (una pestaña nueva basta) abre este `st.dialog` antes de habilitar el chat. Sin aceptar, el campo de mensaje permanece deshabilitado ("Acepta el aviso de datos para poder escribirme").

Caso 1 — Gestante, consulta directa: pregunta informativa que continúa hacia riesgo medio:

The screenshot shows the Maternas application interface. On the left is a sidebar with navigation links: Chat, Dashboard, Documentos, Métricas, Configuración, and Consola. The main area is titled "Maternas — Asistente de Salud Materna" and includes the subtitle "Respondo preguntas sobre embarazo, parto, postparto y lactancia basándome en literatura médica."

The chat history shows a user message: "Tengo hinchazón leve en los pies, estoy en la semana 30, ¿es normal?"

The assistant's response is: "Sí, la hinchazón leve de pies y tobillos es frecuente a partir del tercer trimestre y, en la mayoría de los casos, no indica un problema grave[1]. Mantén las piernas elevadas cuando descanses, usa medias elásticas y evita estar mucho tiempo de pie. Si la hinchazón se vuelve súbita, muy marcada o acompañada de dolor intenso, dificultad para respirar o presión arterial alta, consulta a tu médico o matrona."

Below the response, it lists the source: "Fuentes: [1] GAP_Control prenatal del embarazo normal_6105 - págs. 15-16"

The input field at the bottom contains the text: "Ej: ¿Es normal tener náuseas a las 10 semanas?" and an "Enviar" button.

Consulta real: "Tengo hinchazón leve en los pies, estoy en la semana 30, ¿es normal?" — clasificada `riesgo=Low`, respuesta cálida y natural en español, con cita `[1]` y sin ningún rastro de `<think>` (el hallazgo que motivó

blindar `reasoning_format` en el código, sección 9.3).

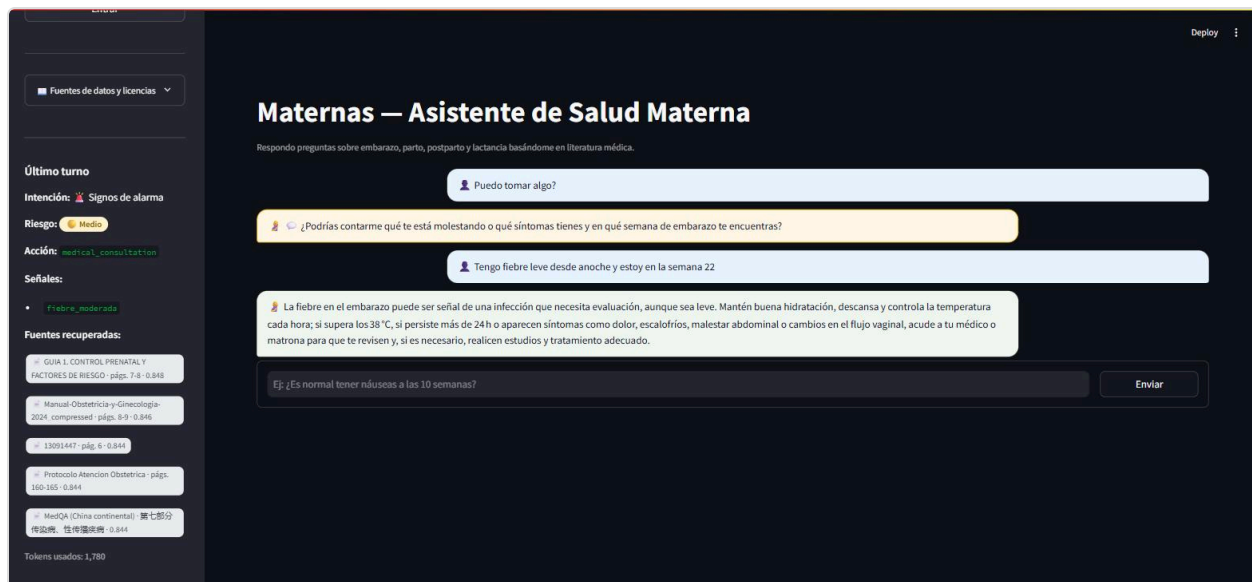
The screenshot shows the Maternas web application interface. On the left is a dark sidebar with navigation options: 'modo administrador', 'Cerrar sesión admin', 'Fuentes de datos y licencias', and 'Último turno'. The main content area has a header 'Maternas — Asistente de Salud Materna' and a subtitle 'Respondo preguntas sobre embarazo, parto, postparto y lactancia basándome en literatura médica.' The chat interface shows a user input: 'Tengo hinchazón leve en los pies, estoy en la semana 30, ¿es normal?'. The system response explains that mild swelling is common in the third trimester and provides advice on leg elevation and medical consultation. Below the response, sources are listed: '[1] GAP_Control prenatal del embarazo normal_6105 - págs. 15-16'. A second user input asks: 'Ahora también tengo fiebre de 37.8 desde ayer, ¿debería preocuparme?'. The system response explains that a temperature of 37.8°C or higher could indicate an infection and lists symptoms to watch for, such as uterine pain or vaginal changes. Sources listed are: '[4] Guía completa. Embarazo. Parto. 2013 - págs. 334, 337, 338'. At the bottom, a third user input asks: '¿Es normal tener náuseas a las 10 semanas?' with an 'Enviar' button.

Mismo hilo, segundo turno: "Ahora también tengo fiebre de 37.8 desde ayer, ¿debería preocuparme?" El panel lateral muestra en vivo: intención `Signos de alarma`, riesgo `Medio`, acción `medical_consultation`, señal `fiebre_moderada` — el sistema combina este mensaje con la hinchazón del turno anterior de la misma sesión.

Caso 2 — Flujo de clarificación, con notificación disparada después de la clarificación (sesión nueva):

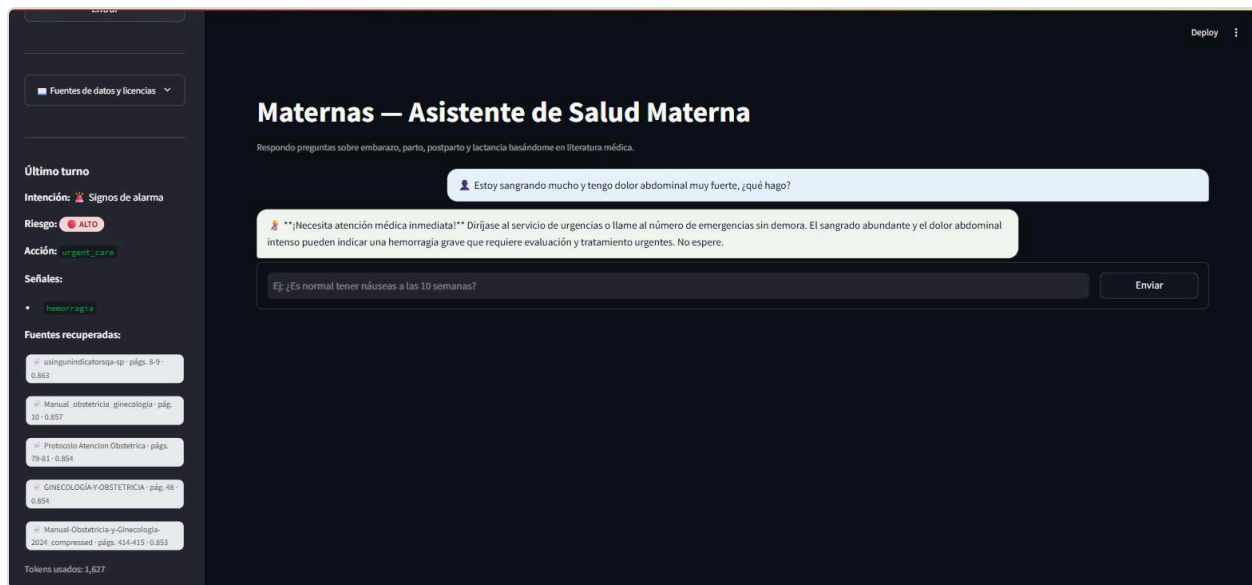
The screenshot shows the Maternas web application interface. The sidebar on the left includes 'Maternas', 'Asistente de salud para madres gestantes', 'API conectada', '253,455 Fragmentos médicos indexados', 'multilingual-es-base', 'Acceso administrador', 'Token de administración', 'Entrar', 'Fuentes de datos y licencias', and 'Último turno'. The main content area has the same header as the previous screenshot. The chat interface shows a user input: 'Puedo tomar algo?'. The system response asks for clarification: '¿Podrías contarme qué te está molestando o qué síntomas tienes y en qué semana de embarazo te encuentras?'. Below this, a third user input asks: '¿Es normal tener náuseas a las 10 semanas?' with an 'Enviar' button.

Consulta real, deliberadamente vaga: "Puedo tomar algo?" (3 tokens, sin keyword de medicamento reconocida). El sistema no genera una respuesta médica a ciegas: pide contexto ("¿Podrías contarme qué te está molestando y en cuántas semanas de embarazo te encuentras?").



Respuesta del usuario a la clarificación: "Tengo fiebre leve desde anoche y estoy en la semana 22." El sistema combina ambos turnos ("Puedo tomar algo" + fiebre/semana 22), clasifica **riesgo=Medio** (señal **fiebre_moderada**) y responde mencionando explícitamente la semana 22 — la pregunta original vaga por sí sola no traía esa información.

Caso 3 — Riesgo alto directo (capa heurística, sin pasar por el LLM de riesgo; sesión nueva):



Consulta real: "Estoy sangrando mucho y tengo dolor abdominal muy fuerte, ¿qué hago?" El panel lateral muestra **riesgo=ALTO**, acción **urgent_care**, señal **hemorragia** (detectada por la capa heurística de **risk_detector.py**, sin consultar al LLM); la respuesta generada por **gpt-oss-120b** sí pasa por el modelo, con el sufijo urgente del prompt (**URGENT_SUFFIX**) y sin fugas de razonamiento.

Observación técnica: el retriever pide $k \times 10$ candidatos internos y filtra a **k=5** finales sobre **medmcqa** + **medqa_*** + **maternaqaes_lm** (Config D, sin BM25). Los scores observados en vivo (0,83–0,86) son consistentes con los reportados en **qa_technical.md** para el mismo tipo de consulta.

6.4 Clasificación de intención

Evidencia visible en las capturas de la sección 6.3 (`intent_classifier.py`) clasifica en el mismo turno que el retrieval). En el Caso 1 la intención pasó de `sintomas_embarazo` (hinchazón) a `signos_de_alarma` (fiebre) al combinarse con el historial; en el Caso 2 la intención se mantuvo en `medicamentos` en la pregunta vaga inicial, y en el Caso 3 se clasificó directamente `signos_de_alarma` .

Observación técnica: `intent_classifier.py` expone 12 categorías fijas con tres niveles de fallback (zero-shot LLM → heurística por keywords → categoría por defecto), garantizando que el sistema siempre devuelva un intent válido incluso sin conexión a Groq.

6.5 Clasificación de riesgo

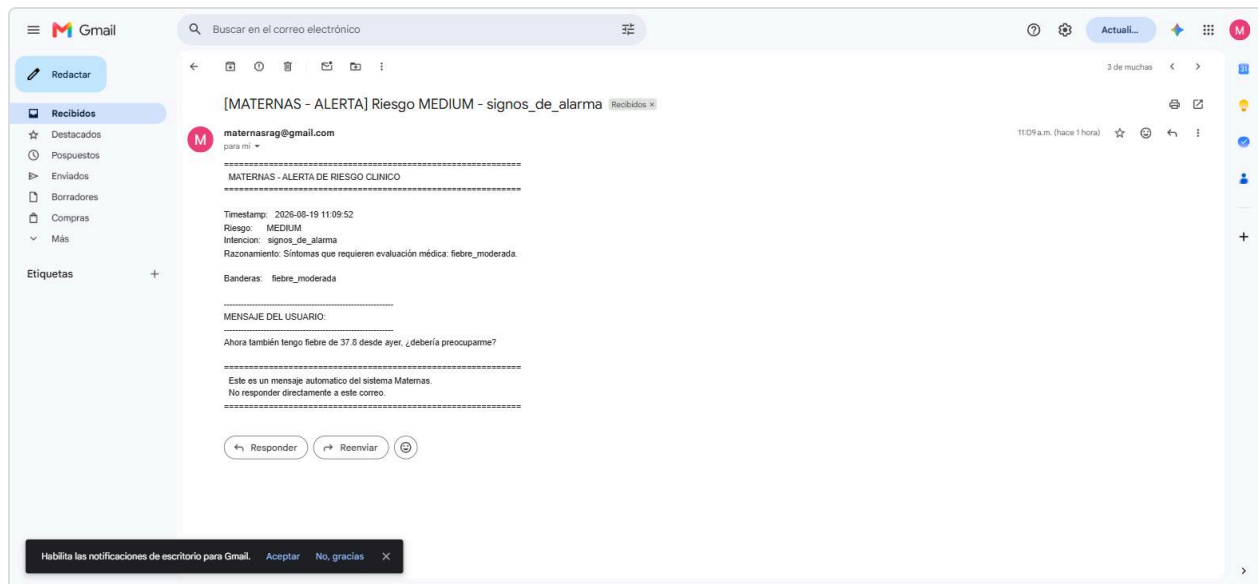
Misma evidencia visual (sección 6.3). El hallazgo relevante de la prueba en vivo se documenta en la sección 9.1: `detect_risk()` combina explícitamente los síntomas mencionados en turnos previos de la conversación con el mensaje actual antes de aplicar la heurística, para no evaluar cada síntoma aislado del cuadro clínico completo. Esto se observó en el Caso 1 (la fiebre se evaluó como continuación del turno de hinchazón) y en el Caso 2 (la clarificación combinó "Puedo tomar algo" con la fiebre y la semana de gestación de la respuesta).

6.6 Redacción

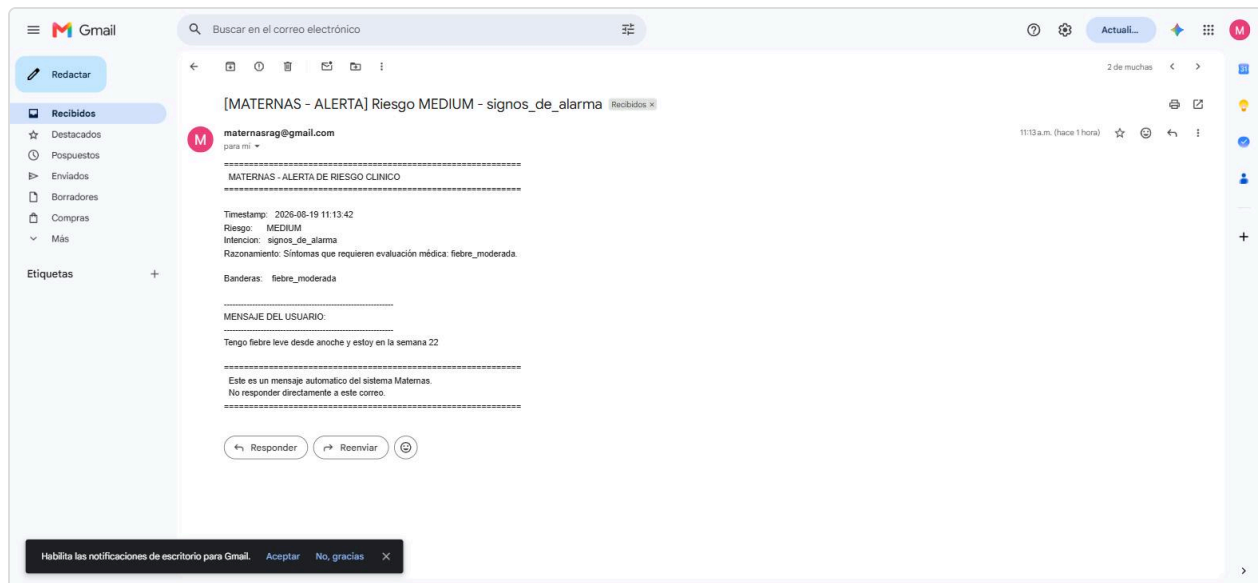
La generación de respuesta y el armado de citas (`citations.py`) se observan en todas las capturas anteriores: el LLM inserta marcadores `[n]` dentro del texto y `build_reference_block()` arma el bloque final `Fuentes:` agrupando por documento con nombre legible y páginas, no "fragmento [n]" genérico (ver Caso 1, sección 6.3). El `reasoning_format:"hidden"` (sección 9.3) evita que el razonamiento del modelo se filtre a este texto.

6.7 Notificador de riesgo (SMTP)

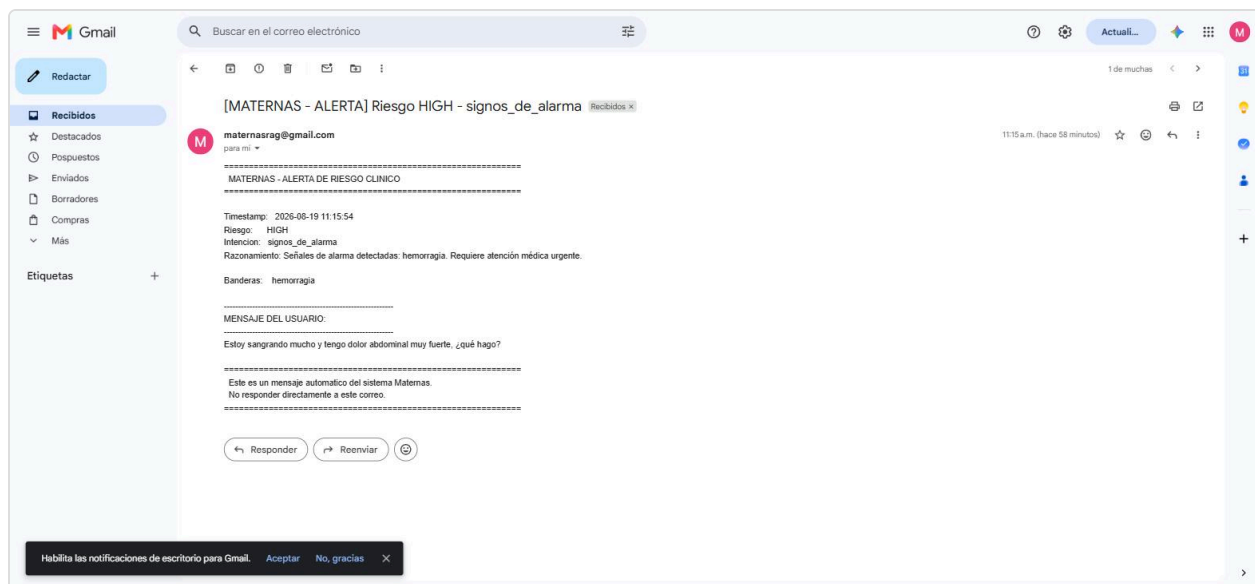
Cada vez que `risk_detector.py` devuelve nivel `medium` o `high` , `NotifierSkill` dispara un correo de alerta real. Se verificó end-to-end contra la bandeja de entrada real (`maternasrag@gmail.com`), incluyendo el caso específico de una notificación disparada **después** de un intercambio de clarificación (Caso 2, sección 6.3) — la ruta con mayor riesgo de la migración, ya que la decisión de notificar pasa por el mismo LLM que ahora razona antes de responder (sección 9.3).



Correo real recibido en maternasrag@gmail.com para el Caso 1 de la sección 6.3, con el mensaje de fiebre citado textualmente y el razonamiento generado por el LLM.



Correo real correspondiente al Caso 2: el "Mensaje del usuario" que se cita es la respuesta a la clarificación ("Tengo fiebre leve desde anoche y estoy en la semana 22") — confirmando que la notificación se disparó una vez el sistema tuvo el cuadro completo, no con la pregunta vaga inicial.



Correo real correspondiente al Caso 3 (hemorragia): riesgo **HIGH**, señal **hemorragia**, notificación disparada de forma incondicional (todo riesgo alto notifica siempre, sin pasar por la decisión del LLM).

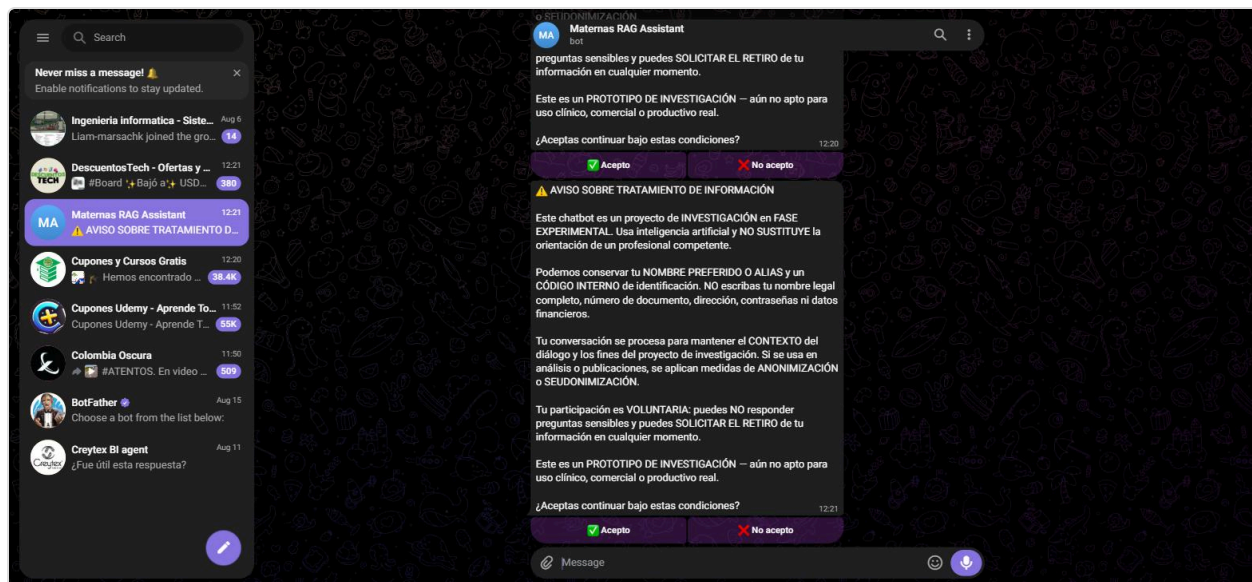
Observación técnica: los tres correos llegaron en menos de un minuto desde el envío del mensaje en cada caso, confirmando que la decisión de notificación (`_run_notification()`), con `max_tokens` subido de 10 a 512 tras la migración — sección 9.3) sigue funcionando de forma confiable con el modelo de razonamiento.

6.8 Streamlit

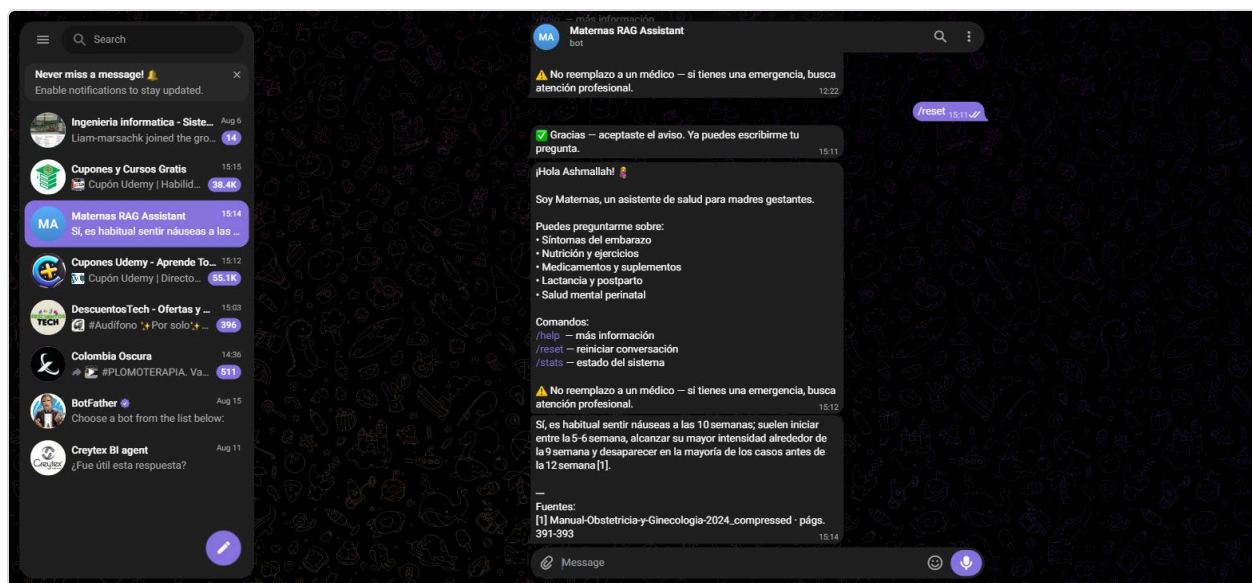
Ver captura de consentimiento en la sección 6.3. La respuesta se transmite token a token vía `POST /chat/stream` (NDJSON: `status` → `meta` → `delta` *→ `done`), visible en vivo como los mensajes de progreso "Analizando tu mensaje..." → "Buscando en la base de conocimiento médico..." → "Redactando la respuesta..." antes de que el texto final aparezca, según `STAGE_LABELS` en `chat_view.py`. Con `gpt-oss-120b` el streaming se probó explícitamente en el Caso 3 de la clasificación de riesgo (ver smoke test, sección 9.3): los eventos `delta` llegan limpios, sin bloques `<think>` intercalados.

6.9 Telegram

Bot real (`@MaternasAssistant_bot`) probado en vivo vía Telegram Web, con la API de producción corriendo en `localhost:8080` y el bot administrado como subprocesso desde el panel admin (sección 7.6).



Enviando `/reset` se reinicia la conversación y el aviso de tratamiento de datos se muestra de nuevo antes de procesar cualquier mensaje — mismo comportamiento que Streamlit, pero como botones inline (`✓ Acepto` / `✗ No acepto`) en vez de un `st.dialog` .



Consulta real: "Es normal tener nauseas a las 10 semanas?" — clasificada `risk=Low` , respuesta cálida en texto plano con bloque `Fuentes:` , sin fugas de `<think>` en el canal de Telegram.

Observación técnica (hallazgo operativo, no atribuible al cambio de modelo): durante esta sesión de pruebas el subproceso del bot se detuvo solo (`proceso terminado, code=1` , visible en los logs de la Consola admin) después de ~33 minutos corriendo, sin traza de excepción capturada en los últimos 200 renglones de log expuestos por el panel. Se reinició sin incidentes desde el botón "Iniciar" (sección 7.6) y continuó funcionando con normalidad. No se investigó la causa raíz a la fecha de este informe — se documenta como observación, no como parte del alcance de la migración de modelo, y queda como ítem de seguimiento.

6.10 Uso en tiempo real, anonimizado (`src/api/usage_sessions.py`)

Subsección nueva del panel de Métricas (ver también 7.4): cuenta cuántas sesiones están activas ahora mismo en Streamlit y en Telegram, cada una con su duración y sus tokens consumidos, **sin ninguna forma de diferenciar una sesión de otra** entre dos lecturas del panel.

`touch(session_id, platform, tokens_used)` se llama en cada turno de `POST /chat` y acumula tokens contra un registro en memoria, sin persistencia. `session_id` lo genera el cliente (`uuid4` aleatorio) — Streamlit lo crea una vez por sesión de navegador (`app.py`), Telegram uno por `chat_id` de conversación pero nunca lo expone; ninguno de los dos se deriva de un identificador real. `active_by_platform()` descarta las sesiones sin actividad hace más de `IDLE_TIMEOUT_SECONDS` (15 min), agrupa por plataforma, ordena por duración descendente y devuelve únicamente `{active_seconds, tokens_total}` por fila — el `session_id` nunca sale de `usage_sessions.py`, ni siquiera hacia `GET /admin/usage_sessions`.

`metrics_view.py::_render_usage_platform()` renderiza esas filas en una tabla sin columna de índice estable, para que tampoco se pueda inferir "la fila 2 de hace un minuto es la fila 1 de ahora".

Observación técnica: el mismo patrón (`session_id` aleatorio, registro en memoria, poda por inactividad de 15 min) se reutilizó para la deduplicación de notificaciones de riesgo de la sección 6.11 — ambos módulos comparten la noción de "sesión activa" pero se definen por separado (`usage_sessions.py` en `src/api/`, `risk_episodes.py` en `src/rag/`) para no romper la regla de capas del proyecto: `src/rag/` no puede importar de `src/api/`.

6.11 Notificador de riesgo: secuencia completa en el correo y fin de la "contaminación" entre turnos

Dos problemas encontrados tras la migración a `gpt-oss-120b` (sección 9.3), corregidos juntos porque comparten la misma causa raíz — el riesgo se venía evaluando turno a turno sin memoria de qué ya se había notificado:

1. **El correo solo mostraba el mensaje que disparó la alerta**, sin el resto de la conversación que le dio contexto clínico. `chain.py` ahora arma la secuencia completa (`history` + mensaje actual, capada a `NOTIFICATION_HISTORY_CAP=20` turnos) y se la pasa a `notify_risk(..., conversation=[...])`. `_build_email_body()` (`src/skills/notifier/tool.py`) imprime cada turno con su rol (`[Paciente]` / `[Asistente]`) y marca el último con `<<< MENSAJE QUE DISPARO LA ALERTA`, para que quede claro cuál de todos gatilló el correo sin perder el resto del cuadro.
2. **Turnos posteriores, ya sin relación con el riesgo detectado, seguían disparando el mismo correo** — el riesgo "medio" de un turno viejo contaminaba la clasificación de mensajes nuevos no relacionados. La solución tiene dos mitades independientes: `risk_detector.py` deja de combinar incondicionalmente el mensaje actual con todo el historial (solo lo hace si el nuevo mensaje sigue siendo parte del mismo cuadro; si no, evalúa el mensaje aislado); y `src/rag/risk_episodes.py` (nuevo) guarda, por `session_id`, únicamente la última señal *efectivamente notificada* (nivel + categorías de banderas — nunca texto del mensaje), para decidir si un turno nuevo es la misma alerta ya enviada o un riesgo distinto que sí amerita un correo nuevo.

`risk_episodes.py` expone una API de dos fases a propósito: `is_new_signal()` (solo lectura, decide si vale la pena seguir evaluando) y `commit()` (se llama únicamente cuando el correo se envió de verdad). Un riesgo

"medio" pasa por una decisión adicional del LLM antes de confirmarse — si `commit()` se llamara antes de esa decisión, un medio que el LLM termina descartando quedaría registrado como si sí se hubiera avisado, y una repetición real de ese mismo riesgo después se suprimiría sin que nunca se hubiera mandado el primer correo. Un turno "low" cierra el episodio de inmediato (`register_low()`): la siguiente alerta medium/high se trata siempre como nueva, sin importar qué banderas comparta con la anterior.

Validación: suite dedicada (`tests/test_risk_episodes.py`, `tests/test_chain_notification.py`, `tests/test_notifier.py`, más casos añadidos a `tests/test_risk_detector.py`) y verificación en vivo durante el desarrollo contra el flujo real (API + SMTP real a `maternasrag@gmail.com`): un turno de riesgo medio dispara un correo con la secuencia completa; un turno siguiente sin relación clínica (p. ej. "gracias") deja de disparar un segundo correo; un turno con un riesgo nuevo y distinto sí dispara uno nuevo. Esta ronda de verificación en vivo no se repitió para esta revisión del informe y no tiene capturas nuevas asociadas (ver nota al inicio de la sección 6); queda cubierta por la suite automatizada (sección 6.13).

6.12 Segunda capa de aceptación: Términos y Condiciones (`src/consent.py`, `src/ui/consent_gate.py`)

Tras el aviso de tratamiento de datos (ya existente, sección 6.3) se añadió una segunda pantalla obligatoria y secuencial: Términos y Condiciones de uso. Ambas capas viven en `src/consent.py`, compartido entre Streamlit y el bot de Telegram, y ambas deben quedar en `"accepted"` antes de habilitar cualquier página o comando — rechazar cualquiera de las dos termina la sesión y la próxima vez que el usuario escribe se le vuelve a mostrar la capa 1 desde cero, nunca a mitad de camino.

En Streamlit, `consent_gate.py::enforce_consent()` corre antes de `st.navigation()` y detiene el script con `st.stop()` mientras falte cualquiera de las dos aceptaciones — a diferencia de la versión anterior (que solo bloqueaba el formulario de envío), así ninguna página del panel (Documentos, Métricas, Configuración, Consola) queda alcanzable detrás del modal. En Telegram, `maternas_bot.py` replica el mismo estado de dos capas (`consent_status`, `terms_status`) con botones inline (☒ Acepto / ☒ No acepto) y un `terms_callback()` independiente del de consentimiento de datos.

El texto de `TERMS_TEXT` se marca explícitamente como **"BORRADOR — PENDIENTE DE REVISIÓN LEGAL"** (texto completo en el anexo 11.4): cubre naturaleza del servicio (prototipo de investigación, no un dispositivo médico), ausencia de garantías, límite de responsabilidad, uso aceptable, propiedad del contenido generado y posibilidad de cambios mientras el proyecto esté en fase experimental. Es una especificación funcional razonable, consistente con el resto del proyecto, no un documento legal definitivo — el propio README (sección "Advertencia de uso y puesta en producción") ya exige su revisión y aprobación por las instancias jurídicas y éticas del proyecto antes de cualquier uso productivo.

Observación técnica: no se generó una captura nueva de este flujo para esta revisión (el intento de verificación visual con el navegador falló por desconexión de la extensión de Chrome durante la sesión de trabajo); el flujo se verificó visualmente durante el desarrollo de la funcionalidad, antes del commit `754ef4a`. El comportamiento de los dos gates (Streamlit y Telegram) está cubierto además por la suite automatizada (sección 6.13).

6.13 Verificación de regresión (suite automatizada)

Suite completa (`pytest`) en verde al commit `754ef4a` : **280 passed, 0 failed** (280/280, frente a los 241/241 reportados en la sección 9.3 al momento de la migración de LLM — el crecimiento corresponde a los tests nuevos de las secciones 6.10–6.12 más los ya existentes). Confirma que ninguna de las tres entregas de esta revisión rompió comportamiento previamente probado (clasificadores, gestión de documentos, config editable, supervisor del bot, citas, chat streaming).

7. Panel de administración

Probado en vivo, con `ADMIN_API_TOKEN` real. El panel vive dentro de la misma app de Streamlit y solo aparece si la sesión se autenticó como admin.

7.1 Acceso

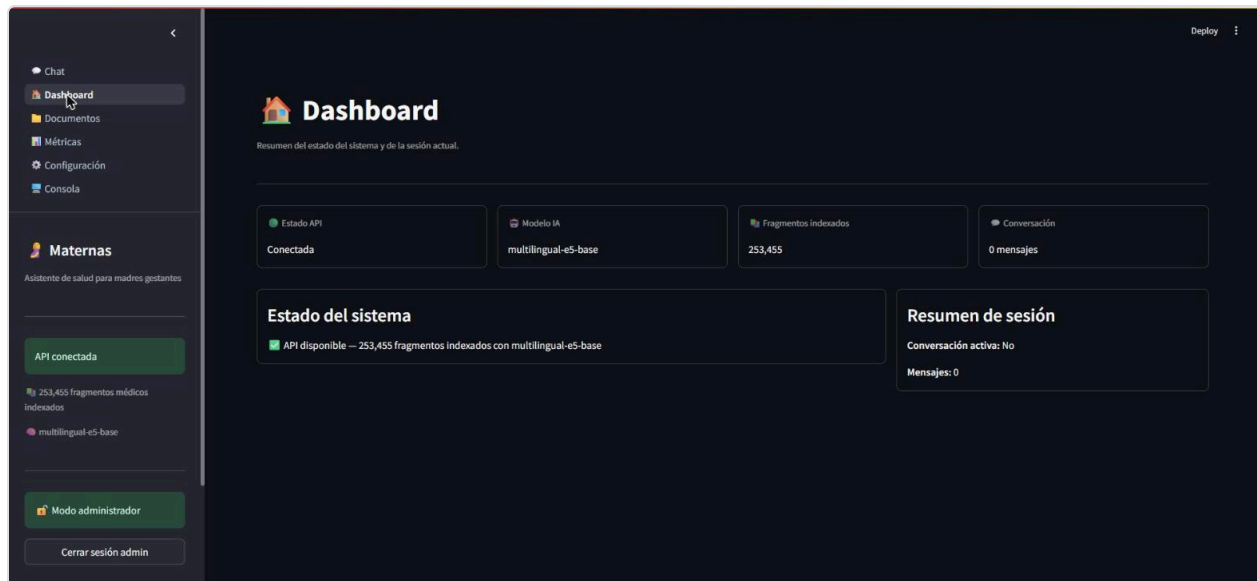


Campo "Token de administración" en la barra lateral, con el token real ya ingresado (enmascarado) — sin token correcto, el panel permanece inalcanzable.



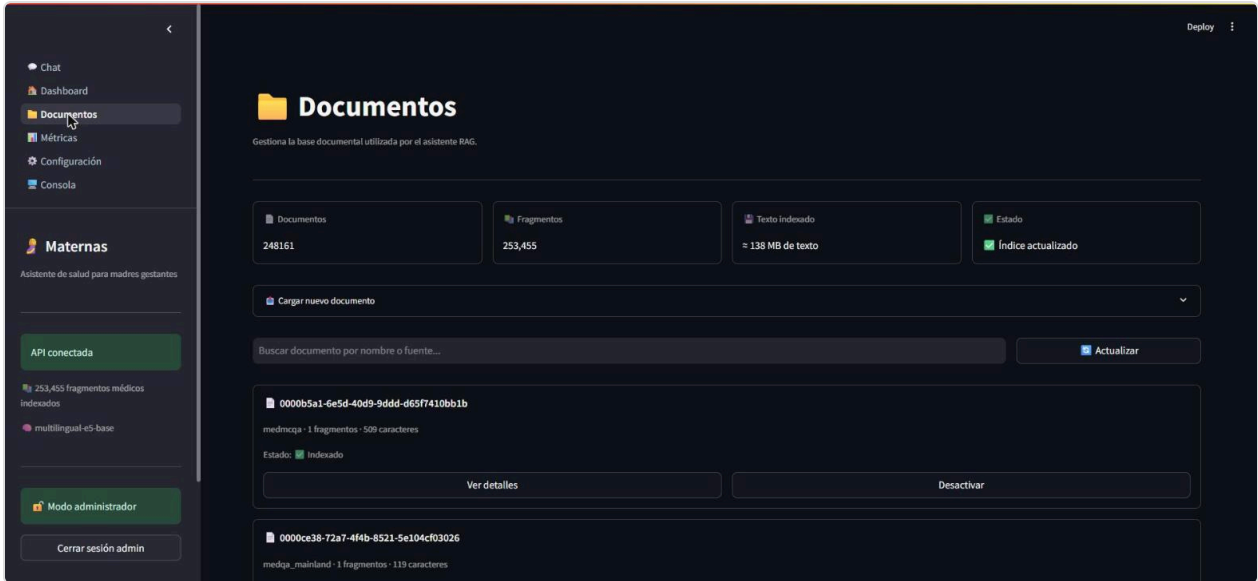
Tras pulsar "Entrar", la navegación lateral revela cinco páginas nuevas: Dashboard, Documentos, Métricas, Configuración y Consola.

7.2 Dashboard



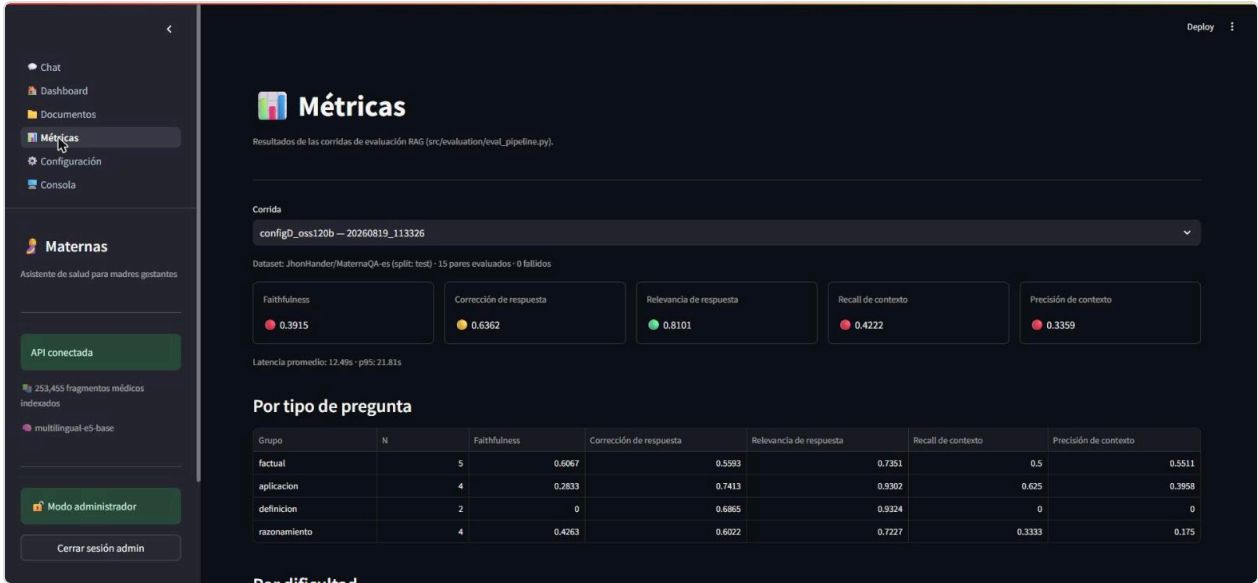
Resumen de sesión y estado del sistema: API conectada, 253.455 fragmentos, modelo de embedding activo y contador de mensajes de la sesión actual.

7.3 Documentos



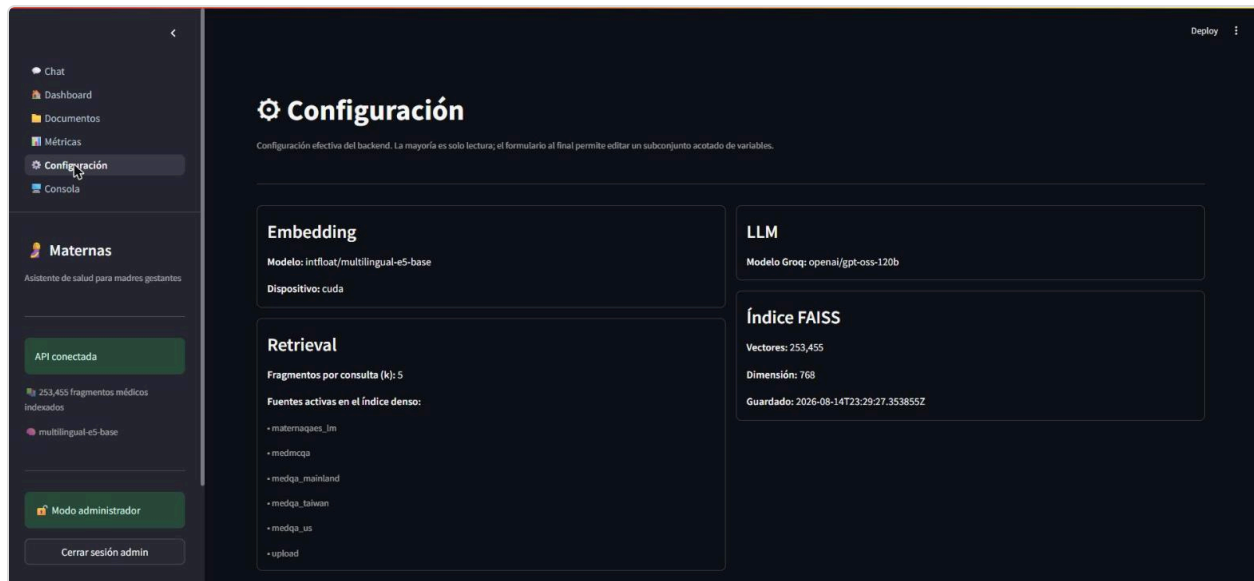
Vista agregada real del índice: 248.161 documentos, 253.455 fragmentos, ~138 MB de texto indexado. Permite buscar, ver detalle paginado, activar/desactivar y subir documentos nuevos — sin cambios respecto a la migración de modelo, el índice de retrieval es independiente del LLM generador.

7.4 Métricas



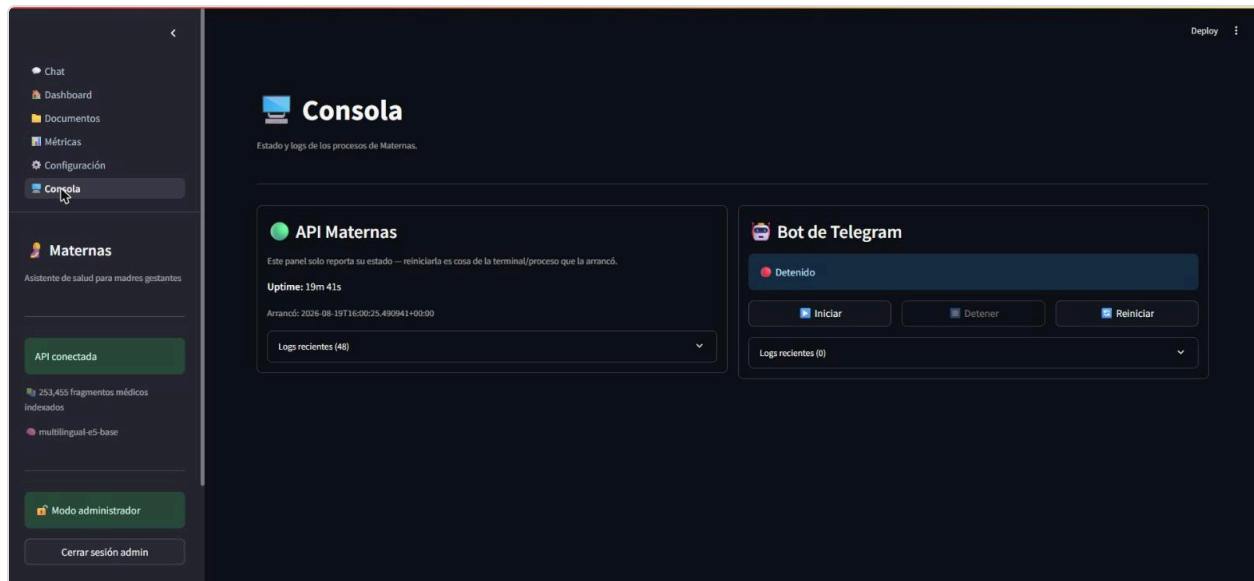
Lectura en vivo de la corrida generada para este informe (`configD_oss120b - 20260819_113326`) sin recomputar nada — coincide exactamente con la tabla de la sección 8 y con `evaluation_reports/eval_report_configD_oss120b_20260819_113326.md` . Esta captura es previa a la subsección "Uso en tiempo real" (sección 6.10), añadida arriba de esta tabla en la misma página tras el 19/08/2026 — sin captura propia en esta revisión.

7.5 Configuración

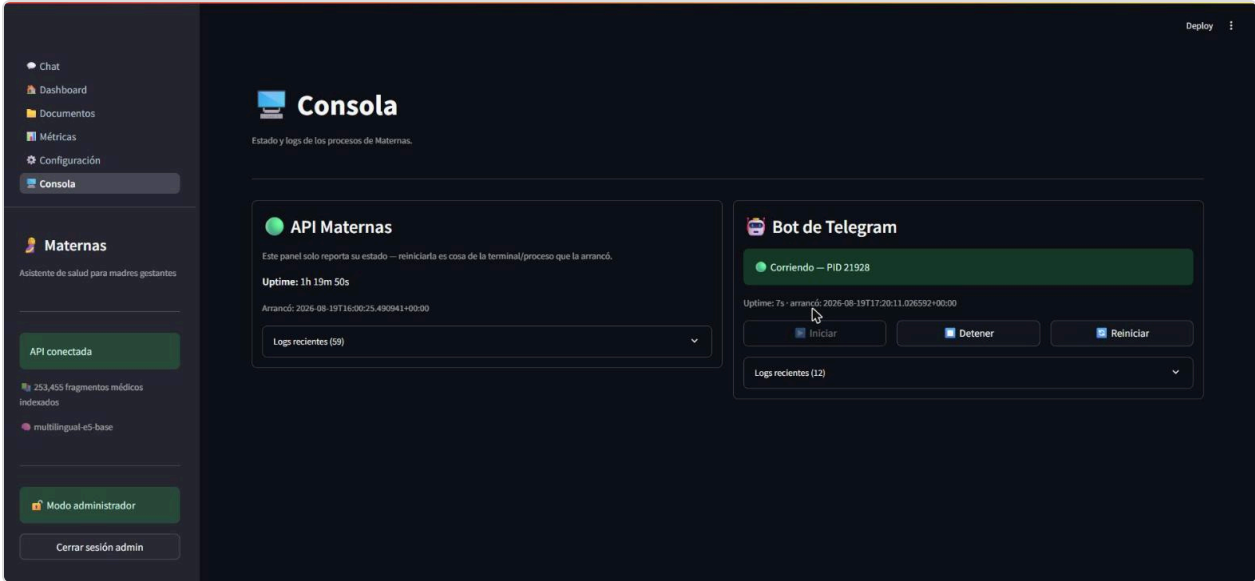


Confirma en vivo la configuración activa: embedding `multilingual-e5-base` en `cuda`, LLM `openai/gpt-oss-120b`, y las fuentes activas del índice denso (`maternaqaes_lm`, `medmcqa`, `medqa_mainland`, `medqa_taiwan`, `medqa_us`, `upload`) — Config D, sin `textbook` ni `multiclinsum`. Esta captura es la confirmación en vivo de que la migración de la sección 9.3 quedó efectivamente desplegada, no solo cambiada en código.

7.6 Consola — API y bot de Telegram



Estado inicial de la sesión de pruebas: API corriendo, bot de Telegram detenido — se inició deliberadamente desde este panel (en vez de manualmente) para poder demostrar el ciclo completo Iniciar/logs con el `.env` de este informe.



Tras pulsar "Iniciar", la API lo lanza como subprocesso hijo (`PID 21928` en la captura) — este es el segundo arranque de la sesión: el primero (`PID 19524`) se detuvo solo tras ~33 minutos (ver observación en la sección 6.9) y se reinició sin incidentes desde este mismo botón.

Observación técnica: el estado del bot que reporta este panel es el que administra `bot_supervisor.py` (`subprocess.Popen`), no cualquier proceso de `maternas_bot.py` corriendo en la máquina — un bot iniciado manualmente por fuera de la API aparece como "Detenido" en este panel hasta que se administra a través de él, consistente con el diseño de estado por-proceso documentado en la sección 9.4.

8. Resultados y evaluación

Framework: **Ragas**, benchmark **MaternaQA-es** (split test, 328 pares QA en español), evaluación en dos fases con modelos independientes (Groq `openai/gpt-oss-120b` genera, Cerebras `gemma-4-31b` evalúa) para evitar que el mismo modelo se autoevalúe.

Nota de migración: esta corrida reemplaza a la del 15/08/2026 (ver `informe-tecnico-maternas-20260815.md`), medida con `llama-3.3-70b-versatile` (N=14). Groq dio de baja ese modelo el 16/08/2026; `openai/gpt-oss-120b` es su reemplazo oficial. Mismo protocolo, mismo seed=42 y mismo tamaño de muestra pedido (15 pares; la corrida anterior tuvo 1 par excluido en fase de evaluación, esta tuvo 0), para que el delta sea atribuible principalmente al cambio de generador.

Métricas de la configuración de producción (Config D, `gpt-oss-120b`)

Métrica	Config D (producción)	Baseline MaternaQA-es (test)	Estado
<code>faithfulness</code>	0,3915	0,7132	●

Métrica	Config D (producción)	Baseline MaternaQA-es (test)	Estado
<code>answer_correctness</code>	0,6362	N/A	
<code>answer_relevancy</code>	0,8101	0,5583	 (supera baseline)
<code>context_recall</code>	0,4222	N/A	
<code>context_precision</code>	0,3359	N/A	
<code>latency_avg_s</code>	12,49 s	—	—
<code>latency_p95_s</code>	21,81 s	—	—

N = 15 pares evaluados, 0 fallidos en generación. Fuente:

[evaluation_reports/eval_report_configD_oss120b_20260819_113326.md](#).

Antes / después de la migración (mismo Config D, mismo protocolo)

Métrica	<code>llama-3.3-70b</code> (15-ago, N=14)	<code>gpt-oss-120b</code> (19-ago, N=15)	Delta
<code>faithfulness</code>	0,497	0,392	▼ -21%
<code>answer_correctness</code>	0,551	0,636	▲ +15%
<code>answer_relevancy</code>	0,733	0,810	▲ +11%
<code>context_recall</code>	0,452	0,422	▪ -7% (ruido)
<code>context_precision</code>	0,360	0,336	▪ -7% (ruido)
<code>latency_avg_s</code>	9,58	12,49	▼ +30%
<code>latency_p95_s</code>	15,93	21,81	▼ +37%

Progresión histórica por configuración

Config	N	Faithfulness	Ans. Correct.	Ans. Relev.	Ctx. Recall	Ctx. Prec.	Lat. (s)
A — FAISS puro	15	0,162	0,350	0,635	0,000	0,000	11,35
B — FAISS+BM25	15	0,228	0,338	0,631	0,000	0,000	10,36
C v3 — +corpus ES	14	0,456	0,532	0,816	0,452	0,388	10,23
D — llama-3.3-70b (15-ago)	14	0,497	0,551	0,733	0,452	0,360	9,58
E — D + HyDE (descartada)	14	0,521	0,500	0,690	0,476	0,337	10,85

Config	N	Faithfulness	Ans. Correct.	Ans. Relev.	Ctx. Recall	Ctx. Prec.	Lat. (s)
D — producción actual (gpt-oss-120b, 19-ago)	15	0,392	0,636	0,810	0,422	0,336	12,49
Baseline MaternaQA-es (test)	—	0,713	—	0,558	—	—	—

Interpretación

- `answer_relevancy` (0,810) **supera el baseline publicado** (0,558) por un margen mayor que antes de la migración, y cruza el umbral 🟢 ($\geq 0,80$) por primera vez en la serie — consistente con que `gpt-oss-120b` es un modelo de razonamiento: el paso de pensar antes de responder parece ayudar a mantenerse enfocado en lo que realmente se preguntó.
- `answer_correctness` sube 15% (0,551→0,636) — el nuevo modelo produce respuestas más cercanas al *ground truth* del benchmark.
- `faithfulness` **baja** 21% (0,497→0,392) — con solo N=15 y un delta de esta magnitud, es plausible que una parte sea varianza estadística ($\sim 0,25$ de desviación estándar según `eval_runbook.md`), pero la dirección es consistente con lo esperado de un modelo que razona más y cita menos literalmente: el juez de Ragas premia afirmaciones verificables palabra por palabra contra el contexto, y un modelo que reformula/sintetiza más (en vez de citar) puntúa peor en esta métrica aunque la respuesta sea igual o más correcta — coherente con que `answer_correctness` subió al mismo tiempo. No se investigó a fondo la causa exacta a la fecha de este informe (ver sección 10).
- `context_recall` / `context_precision` se mantienen prácticamente iguales (delta de ruido) — es la retrieval Config D sin cambios; el juez las computa contra el mismo contexto recuperado en ambas corridas, así que su ligera variación es esperable de la aleatoriedad del propio juez LLM, no del cambio de generador.
- La latencia sube 30–37% — costo esperado del overhead de razonamiento del modelo nuevo, incluso con `reasoning_effort: "low"`.
- El tamaño de muestra (14–15 pares) implica una desviación estándar considerable; los deltas de esta tabla deben leerse como una señal direccional, no una medición de precisión estadística.
- **Pendiente / no medido a la fecha de este informe:** ampliación de la muestra de evaluación a 30 pares para reducir la varianza, y una revisión cualitativa de pares con `faithfulness` bajo para confirmar la hipótesis de "razona/sintetiza en vez de citar" (backlog abierto, ver sección 10).

9. Problemas encontrados y decisiones técnicas

9.1 Riesgo clínico consciente del historial (verificado en prueba en vivo)

`detect_risk()` (`src/classifiers/risk_detector.py`, líneas 330–359) combina explícitamente los síntomas de turnos previos con el mensaje actual antes de aplicar la heurística:

"Cuando se provee `history`, los síntomas mencionados en turnos anteriores de la misma conversación (...) se combinan con el mensaje actual, para no evaluar cada síntoma aislado del resto del cuadro clínico."

Esto se confirmó en vivo dos veces en la sección 6.3: en el Caso 1, una fiebre de 37,8°C reportada en el segundo turno se evaluó junto con la hinchazón mencionada antes en la misma sesión (`riesgo=Medio`, señal `fiebre_moderada`); y en el Caso 2, la clarificación deliberadamente vaga ("Puedo tomar algo?") terminó en `riesgo=Medio` en cuanto la respuesta reveló fiebre y la semana de gestación — la pregunta original vaga, aislada, no traía ninguna señal clínica evaluable. Comportamiento clínicamente conservador e intencional, no un efecto colateral de la UI (se verificó que `chat_view.py` solo lee `meta[-1]`, el último turno; la acumulación de contexto ocurre en el backend).

9.2 Simplificación del bot de Telegram: badge de riesgo removido a propósito

Al preparar la evidencia en vivo de la sección 6.9 se observó que el bot de Telegram envía siempre un único mensaje de texto plano, sin un header de riesgo separado. Revisando el historial de commits del archivo (`git log -p -S "RIESGO ALTO" -- src/bot/maternas_bot.py`) se confirma que esto es una decisión de diseño explícita, no una omisión: el commit `6841c206` ("fix: riesgo clinico acumula contexto entre turnos + citas con ruta de fuente", 6 de agosto de 2026) eliminó deliberadamente el header HTML con badge (`🚨 RIESGO ALTO`, `🟡 Riesgo Medio`, etc.) que el bot enviaba como mensaje aparte. El propio mensaje de commit lo explica: "se quita el badge de riesgo del chat, la urgencia ya queda reflejada en la respuesta" — el texto generado por el LLM ya comunica la urgencia explícitamente (p. ej. "Debes buscar atención médica INMEDIATA..."), por lo que un badge adicional era redundante.

El mismo commit corrigió un bug relacionado con la clarificación: antes, el historial de la conversación **no se guardaba** mientras el sistema esperaba la respuesta a una pregunta de clarificación, causando amnesia total entre turnos — un síntoma mencionado antes de clarificar se perdía en cuanto la usuaria respondía. Esa corrección es, precisamente, la base del comportamiento de riesgo-consciente-del-historial verificado en la sección 9.1 y demostrado en vivo en el Caso 2 de la sección 6.3.

9.3 Migración forzada del LLM generador: `llama-3.3-70b-versatile` → `openai/gpt-oss-120b`

Groq dio de baja `llama-3.3-70b-versatile` el 16/08/2026 (anunciado el 17/06/2026), el LLM de producción del sistema (genera respuestas, clasifica intención, evalúa riesgo, decide notificaciones y redacta clarificaciones). El reemplazo oficial que Groq recomienda, y el adoptado, es `openai/gpt-oss-120b` — se descartó `qwen/qwen3.6-27b` (la otra alternativa sugerida) porque `gpt-oss-120b` ya venía validado como reemplazo directo, con ventana de contexto mayor y sin necesidad de reescribir el cliente Groq (`groq==0.13.1`, sin actualizar, para no arriesgar el workaround de `x_groq.usage` en streaming).

El cambio no fue un simple `GROQ_MODEL=...` en `.env`: `gpt-oss-120b` es un **modelo de razonamiento**, y el código asumía uno que no lo es. Tres consecuencias concretas, encontradas antes de tocar producción:

1. **Fuga de `<think>` a la respuesta.** Groq solo cambia `reasoning_format` a `parsed` automáticamente cuando hay JSON mode o tool use. Los dos clasificadores (`intent_classifier.py`, `risk_detector.py`) usan `response_format={"type": "json_object"}` y se salvan; las 4 llamadas de texto libre en `chain.py`

(`chat()` , `chat_stream()` , `_generate_clarification()` , `_run_notification()`) no. Sin corregirlo, el bloque `<think>...</think>` habría aparecido literalmente en el chat de la gestante.

2. **Los tokens de razonamiento cuentan contra `max_tokens`**. El caso más grave: `_run_notification()` usaba `max_tokens=10` para decidir `"YES"/"NO"` antes de notificar un riesgo medio. El razonamiento se habría comido el presupuesto entero, dejando `content` vacío y `"YES" not in ""` — **un riesgo medio habría dejado de notificar al clínico, en silencio.**

3. **Cascada de fallback.** Si `classify_intent()` recibe `content` vacío, cae a `pregunta_fuera_de_alcance`; `detect_risk()` tiene un atajo que ante ese intent devuelve `low` sin consultar al LLM — un fallo del clasificador se habría convertido en riesgo bajo silencioso.

Solución aplicada (`src/settings.py::groq_reasoning_kwargs()`): un helper gateado por nombre de modelo (`"gpt-oss"` / `"qwen3"` en `GROQ_MODEL`) que agrega `extra_body={"reasoning_effort": "low", "reasoning_format": "hidden"}` en texto libre (sin `reasoning_format` en modo JSON, donde Groq ya fuerza `parsed`), aplicado en los 6 call sites del LLM. Se subieron además los topes de `max_tokens` (10→512 en la decisión de notificación, 100→700 en clarificación, 150→800 en intención, 200→900 en riesgo, 800→1800 en generación) y se blindó el parseo (`content or ""` en vez de asumir no- `None` , con log en `WARNING` si queda vacío) como defensa en profundidad independiente del modelo.

Validación antes de producción: suite `pytest` completa en verde (241/241, sin cambios de comportamiento mockeado); smoke test directo contra los 6 call paths reales de Groq confirmando cero fugas de `<think>`; y finalmente pruebas en vivo — Streamlit y Telegram, sesiones nuevas — que confirmaron tono cálido en español intacto, flujo de clarificación funcionando, y los tres correos SMTP de riesgo (medio, medio-tras-clarificación, alto) llegando con el contenido correcto. El detalle completo de casos y capturas está en la sección 6 de este informe.

9.4 Decisiones de arquitectura (trade-offs)

Decisión	Alternativas consideradas	Razón elegida
FAISS <code>IndexFlatIP</code>	<code>IndexIVFFlat</code> (aproximado)	~253k–380k vectores → búsqueda exacta en 20–50ms; IVFFlat solo se justifica en millones de vectores
<code>multilingual-e5-base</code> (768 dims)	MiniLM, BGE	Soporte nativo ES/EN/ZH en un solo modelo; prefijos obligatorios <code>query:</code> / <code>passage:</code>
Groq <code>openai/gpt-oss-120b</code> para generación (migrado desde <code>llama-3.3-70b-versatile</code>)	<code>qwen/qwen3.6-27b</code> (alternativa sugerida por Groq), GPT-4, LLM local	Groq dio de baja <code>llama-3.3-70b-versatile</code> el 16/08/2026 — cambio forzado, no una comparación libre. <code>gpt-oss-120b</code> es el reemplazo oficial: mismo tier gratuito, mayor velocidad (~500 tok/s) y ventana de contexto (131k). Al ser modelo de razonamiento, requirió <code>reasoning_format:"hidden"</code> y subir los <code>max_tokens</code> en los 6 call sites del LLM para que el presupuesto de razonamiento no vaciara la respuesta ni la decisión de notificación de riesgo medio (<code>src/settings.py::groq_reasoning_kwargs()</code>)

Decisión	Alternativas consideradas	Razón elegida
Cerebras <code>gemma-4-31b</code> como juez Ragas	Groq <code>gpt-oss-120b</code> , <code>llama-3.1-8b</code> (dado de baja, reemplazo <code>gpt-oss-20b</code>)	Sin límite diario de tokens; el juez debe ser independiente del generador en cualquier caso
Chunking ~336 tok para <code>maternaqaes_lm</code>	~879 tok originales	<code>faithfulness</code> 0,133→0,359 (+170%) al re-chunkar — el juez Ragas localiza mejor afirmaciones en fragmentos atómicos
Heurística + LLM en cascada para riesgo	LLM para todo	Heurística: latencia ~0ms, determinismo y sin costo de API para casos obvios (hemorragia, convulsión)
Remoción de <code>textbook</code> / <code>multiclinsum</code> (Config D)	Mantenerlos indexados	Riesgo de licencia no resuelto; impacto en métricas medido primero y confirmado dentro del ruido estadístico
HyDE descartado (Config E)	Adoptarlo en producción	Sin mejora concluyente; +1,27s de latencia y agotamiento de cuota Groq a mitad de la evaluación
Bot de Telegram como subproceso hijo de la API	systemd/Docker/supervisor externo	Proyecto en una sola máquina sin orquestador; suficiente para iniciar/detener/reiniciar desde el panel admin
Badge de riesgo removido del bot de Telegram (commit <code>6841c206</code>)	Mantener el header HTML separado	La urgencia ya queda reflejada explícitamente en el texto de la respuesta del LLM; un badge aparte era redundante (ver sección 9.2)
Uso en tiempo real sin identificador de sesión (<code>usage_sessions.py</code>)	Mostrar un id corto o un índice fijo por fila	Requisito explícito: ninguna fila debe ser distinguible de otra entre dos lecturas del panel — se expone solo duración y tokens, ordenados por duración (sección 6.10)
Dedup de riesgo en memoria, por <code>session_id</code> , sin persistencia (<code>risk_episodes.py</code>)	Persistir episodios en disco/DB	Es una vista de sesión activa, no una métrica histórica — mismo criterio MVP que <code>usage_sessions.py</code> y <code>histories</code> del bot; se pierde al reiniciar el proceso, aceptable (sección 6.11)
API peek/commit en <code>risk_episodes.py</code>	Marcar "notificado" apenas se decide evaluar	Un riesgo medio pasa por una decisión adicional del LLM antes de confirmarse — marcar antes de tiempo suprimiría la repetición real de un riesgo que nunca llegó a notificarse (sección 6.11)

Decisión	Alternativas consideradas	Razón elegida
T&C como borrador marcado "pendiente de revisión legal" , no como texto legal definitivo	Redactar y activar términos definitivos sin revisión externa	El equipo de desarrollo no tiene la competencia jurídica para redactar términos vinculantes; el README ya exige esa revisión antes de producción — mejor un borrador honesto que texto legal no verificado (sección 6.12)
T&C como segunda pantalla secuencial , tras el aviso de datos	Combinar ambos avisos en una sola pantalla	Son dos consentimientos de naturaleza distinta (tratamiento de datos vs. términos de uso); mantenerlos separados permite versionarlos y auditarlos de forma independiente (sección 6.12)

9.5 Limitaciones conocidas

Limitación	Impacto
CORS abierto a <code>*</code>	Cualquier origen puede consumir <code>/health</code> , <code>/chat</code> , <code>/classify</code> (los endpoints admin sí están protegidos)
Cuota de 100k tok/día y 8.000 TPM en Groq (tier gratuito)	Limita evaluaciones largas; con <code>gpt-oss-120b</code> los tokens de razonamiento consumen presupuesto adicional por turno, por lo que el límite por minuto se agota más rápido que con el modelo anterior — mitigado parcialmente con segunda API key
<code>faithfulness</code> 26pp por debajo del baseline	Atribuible a ausencia de los PDFs exactos del benchmark en el índice de producción
<code>--workers 1</code> obligatorio en uvicorn	<code>FAISSStore</code> , sus locks y <code>bot_supervisor</code> son estado por-proceso; múltiples workers divergirían
Sin despliegue automatizado	No hay Dockerfile ni CI/CD; ejecución manual documentada en la sección 5
Sin entrada de voz ni imágenes	El sistema solo acepta texto — ni la UI Streamlit ni el bot de Telegram procesan notas de voz, fotos ni archivos adjuntos
Términos y Condiciones sin revisión legal (sección 6.12)	El texto activo es un borrador funcional del equipo de desarrollo, explícitamente marcado como pendiente de revisión jurídica y ética — no debe tratarse como término vinculante hasta esa revisión
Uso en tiempo real y dedup de riesgo, en memoria y por proceso	Ambos registros (<code>usage_sessions.py</code> , <code>risk_episodes.py</code>) se pierden al reiniciar la API — no hay histórico ni persistencia entre reinicios (mismo criterio MVP que el resto del estado en memoria del sistema)

10. Conclusiones y próximos pasos

Maternas cumple, a la fecha de este informe, con el objetivo funcional definido en el segundo entregable del proyecto: un chatbot RAG que clasifica intención y riesgo, recupera contexto de literatura médica indexada, genera respuestas con citas y escala automáticamente los casos de riesgo medio/alto — verificado en este informe con pruebas reales sobre las tres interfaces (API, Streamlit, Telegram), el panel de administración completo, y confirmación end-to-end del canal de notificación por correo, incluyendo el caso de una notificación disparada después de un flujo de clarificación. La arquitectura de retrieval fue objeto de cinco iteraciones medidas experimentalmente, y las decisiones más recientes (remoción de datasets con licencia no resuelta, descarte de HyDE, simplificación del badge de riesgo en Telegram, migración forzada del LLM generador tras su baja por parte de Groq) siguieron el mismo protocolo: medir o justificar antes de decidir.

Las brechas más relevantes frente al sistema de referencia (`faithfulness` , `context_recall`) tienen una causa raíz identificada y documentada (ausencia de los PDFs exactos del benchmark en el índice de producción, por diseño, para evitar data leakage), no un defecto no diagnosticado. La migración a `gpt-oss-120b` (sección 8) mejoró `answer_relevancy` y `answer_correctness` de forma notable, a costa de `faithfulness` y latencia — un trade-off razonable dado que el cambio de modelo no fue opcional, pero que deja abierta una pregunta concreta para el próximo ciclo: si la caída de `faithfulness` es varianza estadística o un patrón real de "razona/sintetiza en vez de citar textualmente".

Después de esa evaluación, el proyecto sumó tres entregas orientadas a operación responsable más que a calidad de retrieval (secciones 6.10–6.12): visibilidad de uso sin comprometer el anonimato de quien consulta, un notificador de riesgo que deja de saturar al equipo clínico con alertas repetidas sin perder contexto, y una capa explícita de Términos y Condiciones que dice, sin rodeos, que el sistema es un prototipo de investigación y que su texto legal todavía no está aprobado — un paso concreto hacia el requisito de revisión pre-producción que el propio README ya exigía.

Próximos pasos (backlog abierto, `DOCUMENTACION.md` sección 17)

- ☐ Reranker cross-encoder local (`BAAI/bge-reranker-v2-m3`)
- ☐ System prompt más restrictivo ("no tengo información suficiente" explícito)
- ☐ Web search skill (fallback Tavily)
- ☐ Persistencia de historial del bot (SQLite/Redis)
- ☐ Dockerización de API + Streamlit
- ☐ Corpus ampliado (guías OMS, FIGO, guías nacionales adicionales)
- ☐ Ampliar muestra de evaluación a 30 pares para reducir varianza
- ☐ Soporte de entrada por voz o imagen (fuera de alcance actual)
- ☐ Investigar la caída de `faithfulness` post-migración a `gpt-oss-120b` : revisión cualitativa de pares con score bajo, y evaluar si un prompt más explícito sobre citar literalmente compensa la tendencia del modelo a sintetizar
- ☐ **Revisión jurídica y ética del borrador de Términos y Condiciones** (sección 6.12) — bloqueante para cualquier uso productivo, según el README

- ☐ Nueva ronda de evidencia visual en vivo (capturas) para las secciones 6.10–6.12, pendiente por la desconexión de la herramienta de navegador durante esta revisión

11. Anexos

11.1 Endpoints de la API

Método	Ruta	Descripción	Auth
GET	/health	Estado del servicio, vectores cargados, modelo activo	No
POST	/chat	Turno completo RAG	No
POST	/chat/stream	Igual que /chat, NDJSON token a token	No
POST	/classify	Solo clasificadores (intent + risk)	No
GET/PATCH	/documents*	Gestión del índice FAISS en caliente	X-Admin-Token
GET/PATCH	/admin/config	Configuración editable de 10 variables .env	X-Admin-Token
GET	/admin/evaluations*	Lectura de corridas Ragas ya generadas	X-Admin-Token
GET/POST	/admin/bot/*	Control del subprocesso del bot de Telegram	X-Admin-Token
GET	/admin/usage_sessions	Sesiones activas por plataforma (Streamlit/Telegram), anonimizado — sección 6.10	X-Admin-Token

11.2 Variables de entorno relevantes

Variable	Rol
GROQ_API_KEY / GROQ_API_KEY_2	LLM principal / segunda clave para evaluación Ragas
CEREBRAS_KEY	Judge de evaluación Ragas
EMBEDDING_MODEL, EMBEDDING_DEVICE	Modelo y dispositivo de embedding
RAG_TOP_K	Fragmentos recuperados por consulta (default 5)
ADMIN_API_TOKEN	Protege el panel administrativo (fail-closed si vacío)

Variable	Rol
<code>TELEGRAM_BOT_TOKEN</code>	Token del bot (BotFather)
<code>NOTIFIER_*</code>	Configuración SMTP del canal de alertas
<code>STATUS_CHECK_INTERVAL_LOW/MEDIUM/HIGH_SECONDS</code>	Cadencia del scheduler de check-ins según riesgo

11.3 Referencias a documentación completa

- `docs/DOCUMENTACION_oss120b_20260824.md` — documentación técnica completa del sistema, ya con las tres entregas de las secciones 6.10–6.12
- `foragents/qa_technical.md` — 32 preguntas técnicas resueltas, incluyendo las decisiones de licencia (Q28, Q31) y HyDE (Q32) citadas en este informe
- `foragents/retrieval_arquitecturas_configs.md` — detalle de las configuraciones de retrieval A y B
- `evaluation_reports/` — reportes crudos y agregados de todas las corridas Ragas
- `README.md` — sección "Advertencia de uso y puesta en producción", checklist de requisitos antes de cualquier despliegue productivo

11.4 Texto completo del aviso de tratamiento de datos y de los Términos y Condiciones

(`src/consent.py`)

Ambos se muestran, en este orden, al abrir una sesión nueva en Streamlit o en Telegram (sección 6.12). Se reproducen aquí tal como están en el código, sin editar — el segundo está explícitamente marcado como borrador.

Capa 1 — Aviso sobre tratamiento de información:

AVISO SOBRE TRATAMIENTO DE INFORMACIÓN

Este chatbot es un proyecto de INVESTIGACIÓN en FASE EXPERIMENTAL. Usa inteligencia artificial y NO SUSTITUYE la orientación de un profesional competente.

Podemos conservar tu NOMBRE PREFERIDO O ALIAS y un CÓDIGO INTERNO de identificación. NO escribas tu nombre legal completo, número de documento, dirección, contraseñas ni datos financieros.

Tu conversación se procesa para mantener el CONTEXTO del diálogo y los fines del proyecto de investigación. Si se usa en análisis o publicaciones, se aplican medidas de ANONIMIZACIÓN o SEUDONIMIZACIÓN.

Tu participación es VOLUNTARIA: puedes NO responder preguntas sensibles y puedes SOLICITAR EL RETIRO de tu información en cualquier momento.

Este es un PROTOTIPO DE INVESTIGACIÓN — aún no apto para uso clínico, comercial o productivo real.

¿Aceptas continuar bajo estas condiciones?

Capa 2 — Términos y Condiciones de uso (borrador, pendiente de revisión legal):

TÉRMINOS Y CONDICIONES DE USO (BORRADOR — PENDIENTE DE REVISIÓN LEGAL)

Este texto es una especificación funcional preliminar, no un documento legal definitivo. Su contenido debe ser revisado y aprobado por las instancias jurídicas y éticas del proyecto antes de cualquier uso productivo (ver README, sección "Advertencia de uso y puesta en producción").

1. NATURALEZA DEL SERVICIO: Maternas es un prototipo de investigación desarrollado en el marco de la Convocatoria 890 de Minciencias y la Institución Universitaria de Envigado. No es un dispositivo médico, no presta servicios de salud y no sustituye el diagnóstico, tratamiento ni consejo de un profesional competente.

2. SIN GARANTÍAS: El servicio se ofrece "tal cual", en fase experimental, sin garantía de disponibilidad, exactitud, vigencia ni idoneidad para un fin específico. Las respuestas se generan con inteligencia artificial y pueden contener errores.

3. LÍMITE DE RESPONSABILIDAD: El equipo del proyecto no asume responsabilidad por decisiones tomadas a partir de las respuestas del chatbot. Ante cualquier urgencia o duda médica real, contacta a un profesional de salud o a los servicios de emergencia.

4. USO ACEPTABLE: No debes usar este servicio para suplantar a terceros, ingresar datos de otra persona sin su consentimiento, ni para fines distintos a la consulta informativa de salud materna que motiva este proyecto.

5. PROPIEDAD Y CONTENIDO: Las respuestas se generan a partir de fuentes bibliográficas citadas dentro de cada respuesta; su uso está sujeto a la licencia de cada fuente (ver README). El código del proyecto y su documentación pertenecen al equipo de investigación.

6. CAMBIOS: Estos términos pueden actualizarse mientras el proyecto esté en fase experimental; la versión vigente es la que se muestra al inicio de cada sesión nueva.

¿Aceptas continuar bajo estos Términos y Condiciones?

Rechazar cualquiera de las dos capas muestra el mismo mensaje de despedida (`FAREWELL_TEXT`) y termina la sesión; el próximo mensaje vuelve a mostrar la capa 1 desde cero.

Informe generado a partir del código, documentación y evidencia en vivo del repositorio `maternas-rag`. La evaluación en vivo (secciones 6.3–6.9, 7, 8) corresponde al commit `5707d01` (19/08/2026); las secciones 6.10–6.13 y las tablas de decisión asociadas se añadieron en esta revisión de contenido al commit `754ef4a` (25/08/2026), documentadas con el código real y la suite `pytest` (280/280), sin una nueva ronda de capturas de pantalla. Todo dato reportado es trazable al código fuente, la documentación existente, los reportes de `evaluation_reports/`, el historial de git o la evidencia en vivo listada arriba — sin datos inventados.